How Malware Has Been Analyzed and Detected (PMA 101 — Basic Static Techniques)

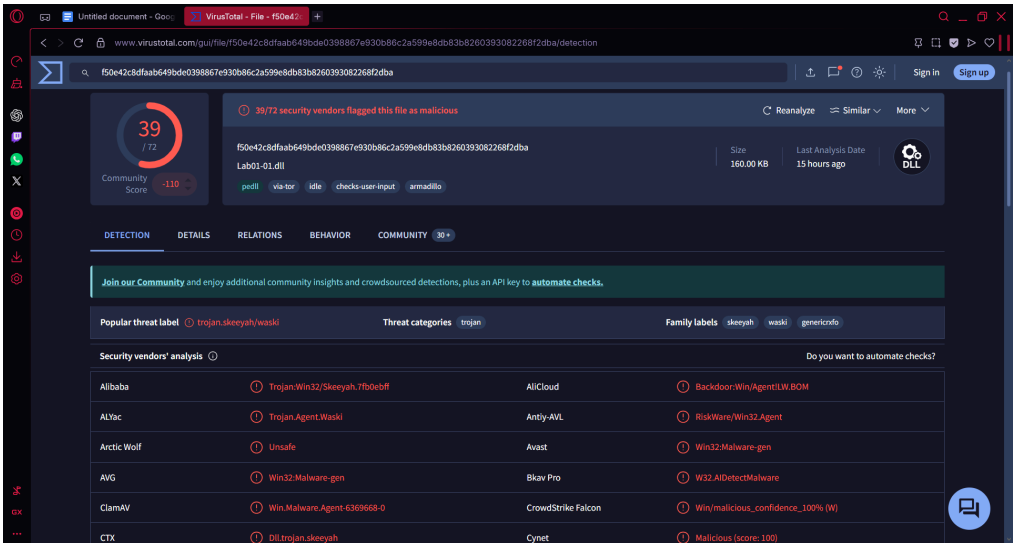
This assignment walks through *static malware analysis*—examining malware **without executing it**—using several common tools. Each tool reveals different characteristics of the malicious files (Lab01-01.exe and Lab01-01.dll). Together, these steps replicate a real-world process of identifying, profiling, and understanding malware samples.

1. Identification Using VirusTotal

Goal: Determine whether the file is already known as malware.

Explain the process:

1) First, we **turned off the device's firewalls, antivirus and protection** to allow the practical malware file to be downloaded without getting alerted or blocked.
2) Then I **downloaded the malware** file which was obtained from **https://github.com/mikesiko/PracticalMalwareAnalysis-Labs**
3) After several file **unzipping** process, we went to Virus Total through **https://www.virustotal.com**
4) **Lab01-01.dll** was submitted and below is the result:



2. Why this works?

1) Virus total works because it compares a file to a database of hundreds of antivirus engines.
2) Allows a quick determination whether the file is already known as malware by the security community. MyCERT (Malaysia Computer Emergency Response Team)
3) The process helps in identifying various threat labels and categorising the malware.
4) * Using the hash value of the sample is a safer method than uploading the file itself, as uploading the file may alert attackers. (good way to use Virus Total, hash | if not the attacker that monitors Virus Total or someone has access to it will notice them and the hacker will know and myb they will change plan to do another attack)

Outcome:

We could see that it has a **Community Level** of **39/100**,

Popular Threat Label: trojan.skeeyah/waski,

Categorised as **Trojan**,

And has several **Family Labels** like **skeeyah** (malware that add backdoor to the system), **waski** (malware that spread through spam emails + malicious ZIP file attachment disguised as PDF) and **genericrxfo** (trojan that establish remote access connections, perform keylogging, collect system info).

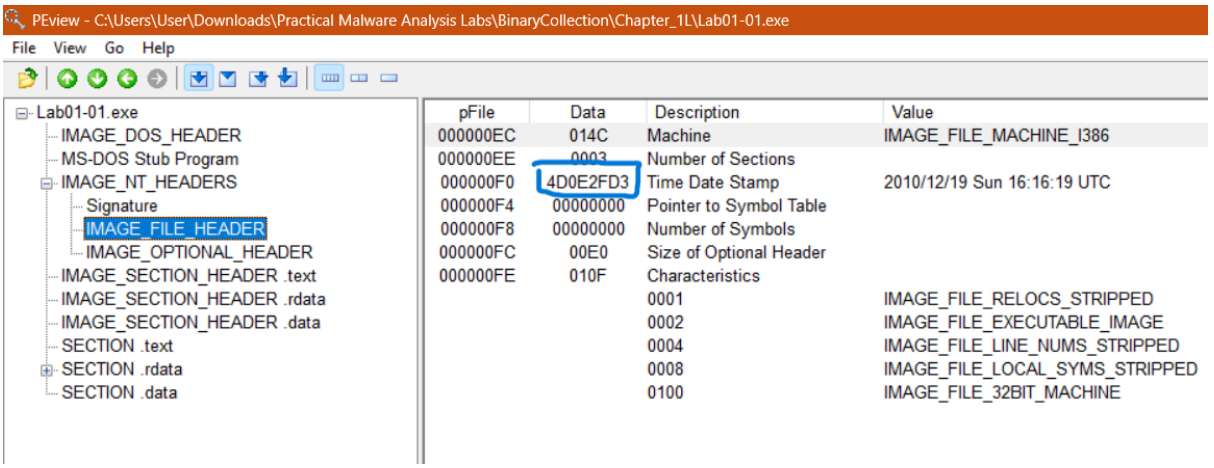
Thus, we can conclude that this file does **indeed contain malware**, specifically a **trojan.**

2. File Structure Analysis Using PEview

Goal: Inspect internal structure of PE (Portable Executable) files.

Process:

1) Downloaded **PEview** from http://wjradburn.com/software/
2) The file was unzipped and then executed.
3) A file explorer in low resolution automatically popped up on the screen.
4) **Lab01-01.exe** was chosen and the **Open** button was clicked on after.
5) Navigated to the left panel, expanded the **"IMAGE_NT_HEADERS"** by clicking on **"+"** next to it.
6) Clicked on **"IMAGE_FILE_HEADER"** and the following would appear:



For **Flag PMA 101.1: Data,** the answer is the boxed line in the **Data** column: **4D0E2FD3.**

Insights Gained:

1) We can view the date when the file was compiled in the Time Date Stamp.

Outcome:

The file was compiled on Sunday, 16:16:19 UTC on 19th December 2010.

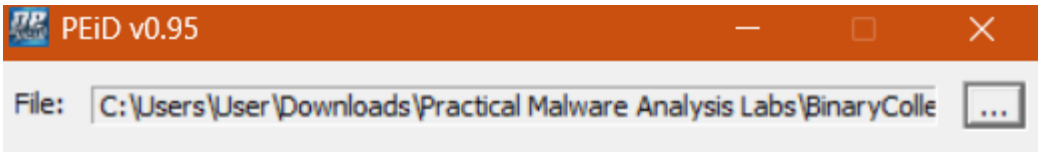
3. Packer / Compiler Detection Using PEiD

Goal: Determine if malware is:

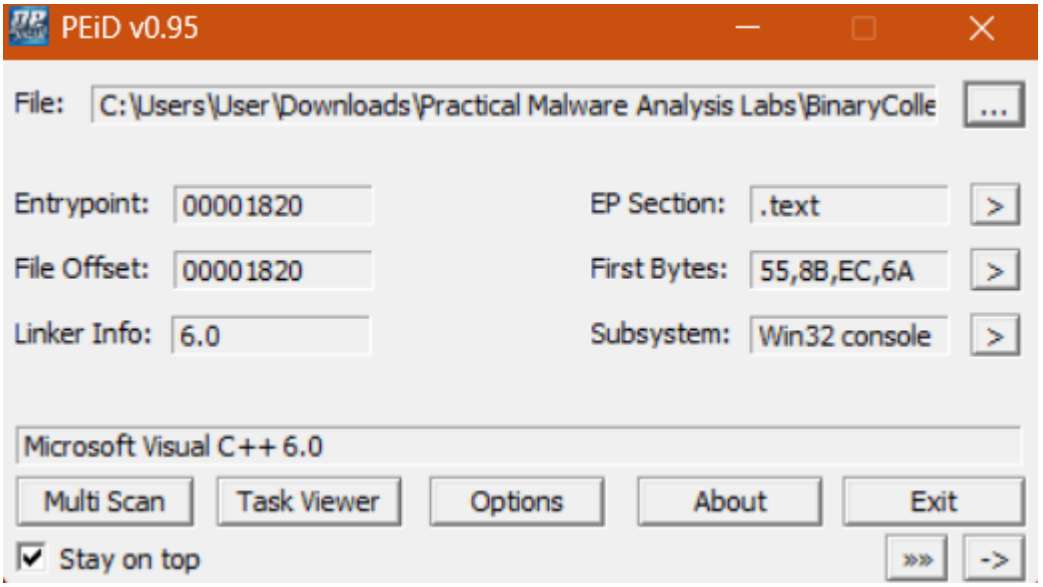
- Packed (compressed/encrypted to evade detection)
- Written in a particular language or compiler

Process:

1) Downloaded **PEiD** via https://bowneconsultingcontent.com/pub/PMA/pma101a/PEiD-0.95-20081103.zip
2) File extracted and executed.
3) Click on the **"..."** button next to the search file bar.



4) Picked **Lab01-01.exe** file and below is the result:



The answer for **Flag PMA 101.2: First Bytes: 55,8B,EC,6A.**

Outcome:

1) PEiD shows what language sample was written in and what was used (if packed).
2) Microsoft Visual C++ refers to what was used to write the file.

4. String Analysis Using BinText

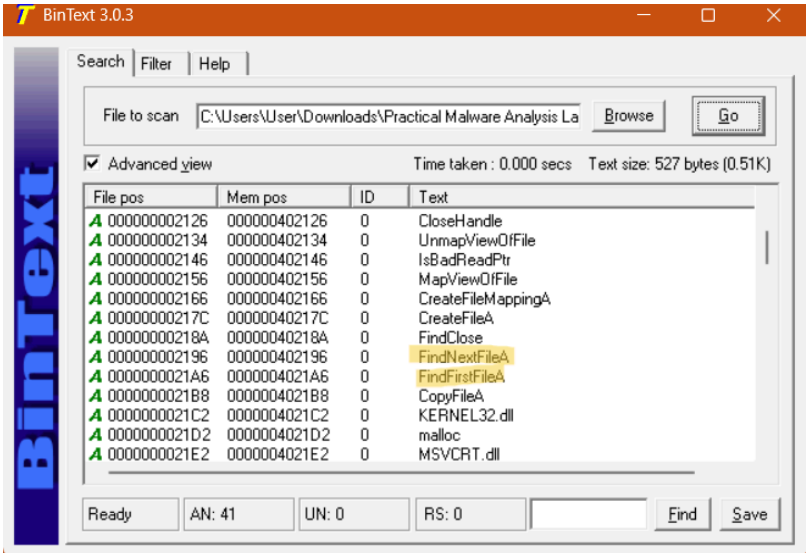
Goal: Extract readable strings from the binaries.

Why Strings Matter:

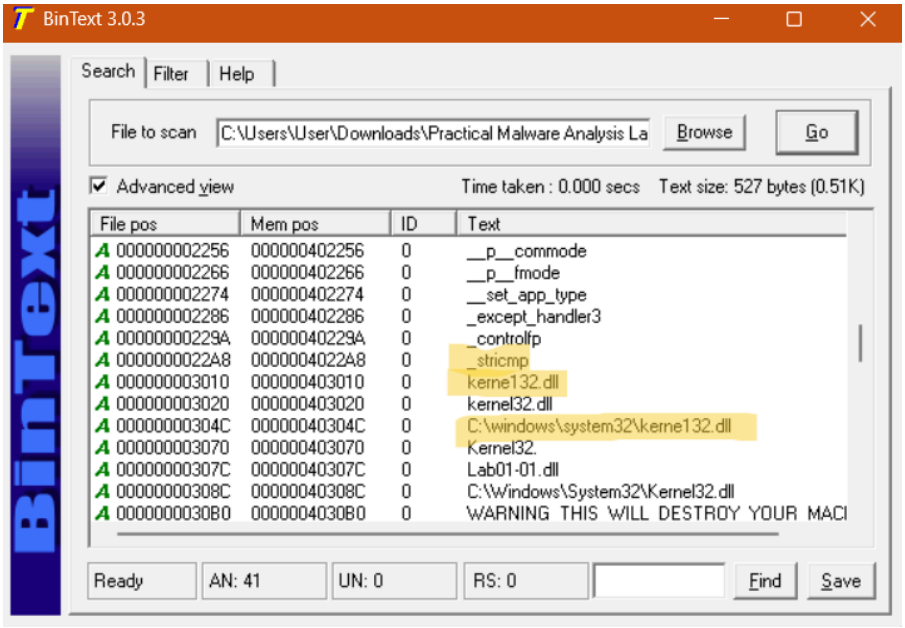
- String often contains human-readable text
- Fastest ways in static analysis to get meaningful information about malware is trying to do.
- Exposed core functionality:
    - URLs/IP Address (Command and Control - C2 Server)?
    - File paths ?
    - Registry key ?

Process for Lab01-01.exe:

1) Downloaded the **BinText** application through [bintext303.zip]
2) Unzipped the file and had it executed.
3) Clicked **"Browse"** and picked **Lab01-01.exe.**
4) Clicked **"Go"** and searched for **FindNextFileA** and **FindFirstFileA.**



5) Scroll further down and find **_stricmp**, **kerne123.dll** and **C:\windows\system32\kerne132.dll**.



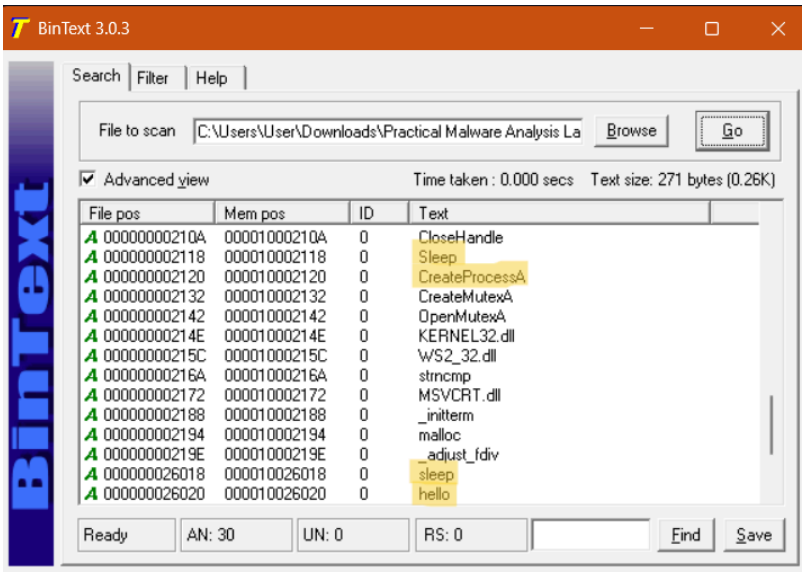
Key Detection Insight:

1) **BinText** is used to view strings (a way to analyze files).
2) **FindNextFileA** and **FindFirstFileA** are Windows API functions used to search through a directory.
3) **_stricmp** is used to compare a string to a desired value.
4) **Kernel132.dll** is a deceptive filename to make the malware look like a Windows system file.
5) **C:\windows\system32\kerne132.dll** is the full path to the malicious file, very likely a useful indicator of compromise.

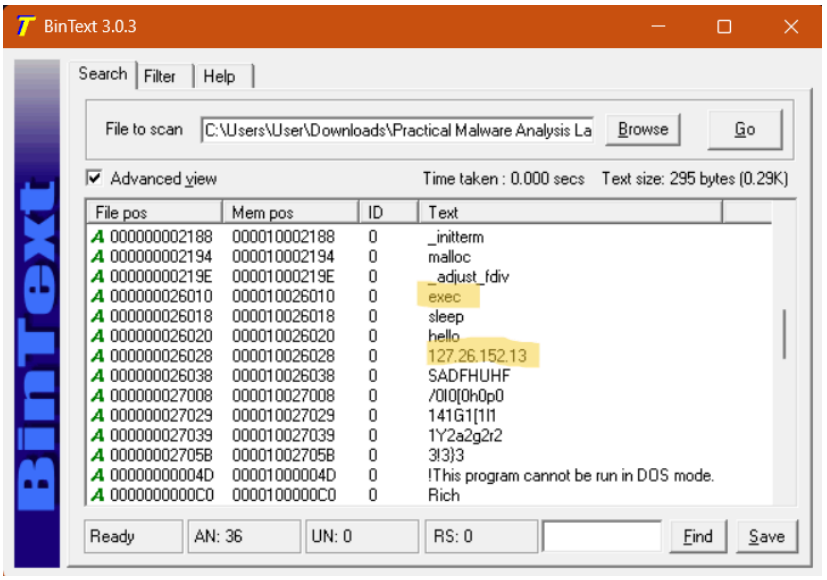
Process for Lab01-01.dll:

Strings reveal:

1) open the **Lab01-01.dll** file and click **Go**.
2) Find **Sleep, CreateProcessA, sleep** and **hello.**



3) Go to **Filter** and adjust **Min Text Length from 5 to 4.**
4) Return to **Search**, click **Go** and the output was as shown below:



5) Exec was located and the answer for **Flag PMA 101.3: IP Address:** 127**.26.152.13.**

Outcome:

1) The difference between **Lab01-01.exe** and **Lab01-01.dll** is the file type.
2) An **EXE file** is a standalone application that can be run directly, while a **DLL file** is a library of shared code used by other programs and cannot be run on its own.

3) **Allow application to show authenticity - wintrust**
4) **Jumpa file not wintrust.dll. That file contains malware**
5) **Sleep** is a Windows API Function used to sleep.
6) **CreateProcessA** is a Windows API function used to launch a program.
7) **sleep** and **hello** are commands that can be sent over the network to tell the malware to sleep, and some functions called “hello”.
8) **exec** is the command to launch the program.

5. API Dependency Analysis Using Dependency Walker

Goal: Determine which Windows API functions the malware imports.

Why API Calls Matter:

Malware relies heavily on;

- Windows API, a massive library of pre-written code that the OS provides for all programs
- Malware relies heavily on these API calls because it cannot steal your data, connect to its C2 server, or achieve persistence **without asking the operating system (Windows) for permission** to perform those actions using specific API functions.

Analyzing Lab01-01.exe

Key Imports from KERNEL32.DLL:

- **FindFirstFileA**
- **FindNextFileA**

These indicate:

File System Access: The malware is not just focused on one single file or task; it is designed to **look around** the computer.

`FindFirstFileA`**:** This function is the starting point. It tells the Windows Operating System (OS) to begin a search for a file or directory that matches a specific pattern (e.g., all `.doc` files, all files with a particular name).

- **Simple View:** *It starts the scouting process.*

`FindNextFileA`**:** Once the first file is found, this function is used repeatedly to continue the search and retrieve the details of the **next matching file** in the directory.

- **Simple View:** *It continues to list the targets.*

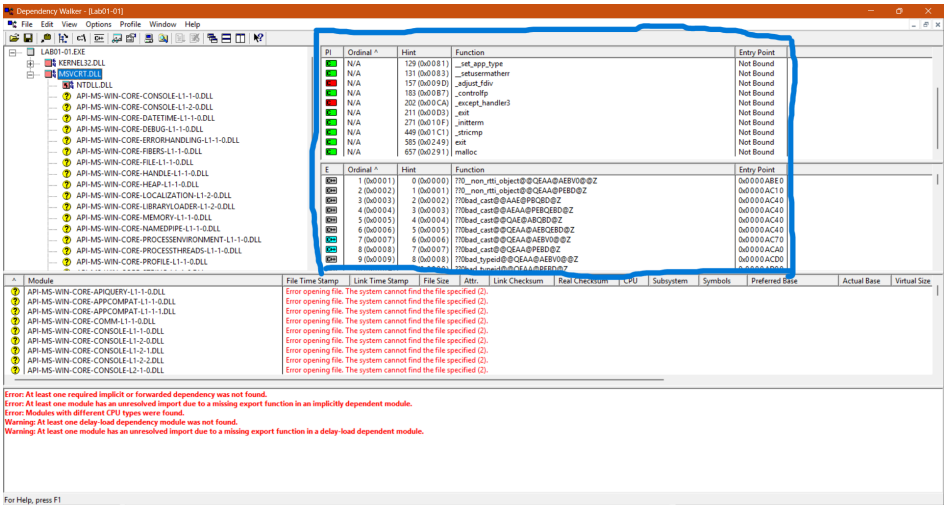
Malware Intent: In the context of malicious software, this capability strongly suggests the malware is designed for activities such as:

- **Information Gathering/Stealing:** Scanning the computer for valuable files (documents, pictures, passwords).

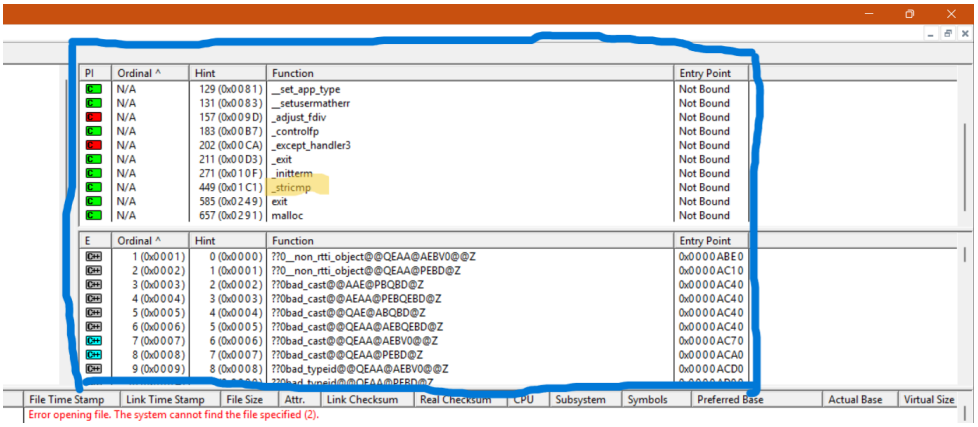
- **Self-Propagation:** Searching for other executable files or network shares where it can copy itself to spread the infection.
- **Target Mapping:** Building a map of the machine's file structure before proceeding with other actions.

Process:

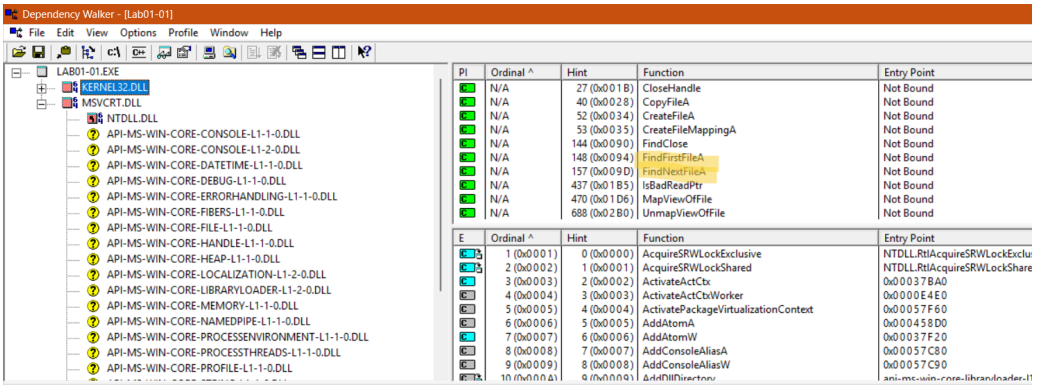
1) Go to http://www.dependencywalker.com/ and download the **Version 2.2.6000 for x64 [468k]**.
2) Unzip the file and run it.
3) Click on the **File** button on the top left corner and choose **Lab01-01.exe.**
4) Click open.
5) On the **Model Dependency Tree View (Top Left Panel)**, click on the **'-' button** next to **KERNEL32.DLL** and **MSVCRT.DLL** will be visible.
6) Click on it and on the right pane will show Parent Imports.



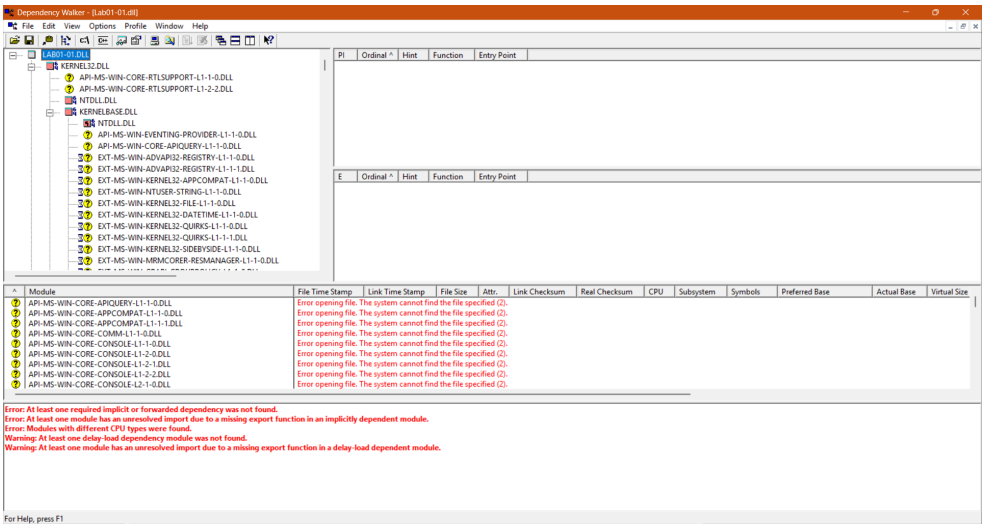
7) Find **_stricmp.**



8) Click on **KERNEL32.DLL** and find **FindNextFileA** and **FindFirstFileA**



9) Open **Lab01-01.dll** and collapse the tree until **WS2_32.DLL** is visible.



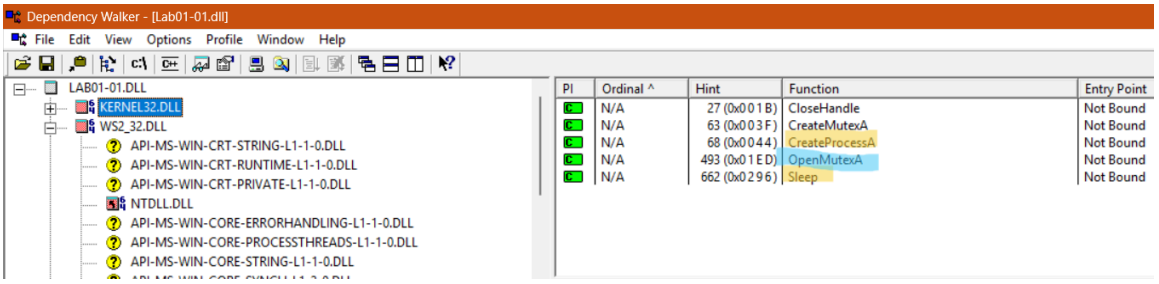
10) Click on it.
11) No Function is observed with Ordinal number visible -> **Linking by Ordinal**.

PI	Ordinal ^	Hint	Function	Entry Point
	3 (0x0003)	N/A	N/A	Not Bound
	4 (0x0004)	N/A	N/A	Not Bound
	9 (0x0009)	N/A	N/A	Not Bound
	11 (0x000B)	N/A	N/A	Not Bound
	16 (0x0010)	N/A	N/A	Not Bound
	19 (0x0013)	N/A	N/A	Not Bound
	22 (0x0016)	N/A	N/A	Not Bound
	23 (0x0017)	N/A	N/A	Not Bound
	115 (0x0073)	N/A	N/A	Not Bound
	116 (0x0074)	N/A	N/A	Not Bound

12) Find **accept, bind** and **connect**.

E	Ordinal ^	Hint	Function	Entry Point
C	1 (0x0001)	161 (0x00A1)	accept	0x00026270
C	2 (0x0002)	162 (0x00A2)	bind	0x00012620
C	3 (0x0003)	163 (0x00A3)	closesocket	0x00010B10
C	4 (0x0004)	164 (0x00A4)	connect	0x0000EE00
C	5 (0x0005)	171 (0x00AB)	getpeername	0x0002A190
C	6 (0x0006)	176 (0x00B0)	getsockname	0x00012410
C	7 (0x0007)	177 (0x00B1)	getsockopt	0x000127C0
C	8 (0x0008)	178 (0x00B2)	htonl	0x00025E00
C	9 (0x0009)	179 (0x00B3)	htons	0x00025E10
C	10 (0x000A)	184 (0x00B8)	ioctlsocket	0x0000E860

13) Click on **KERNEL32.DLL**, find **CreateProcessA** and **Sleep**.



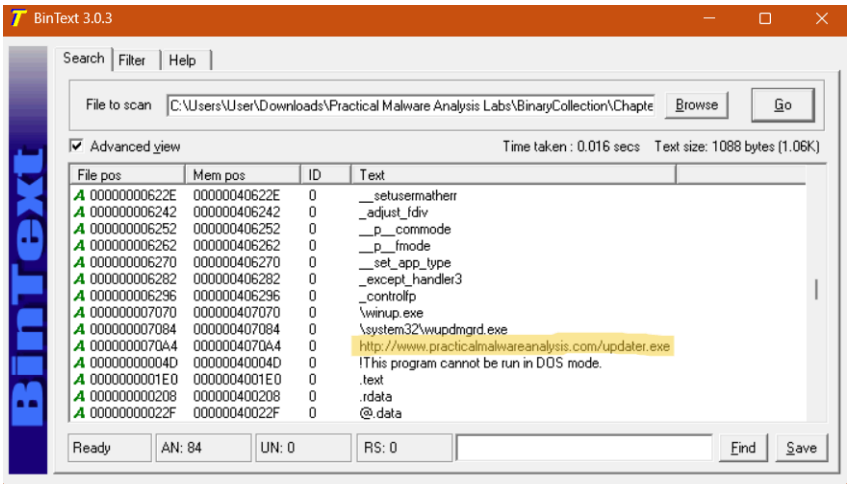
Flag PMA 101.4: Function Name: OpenMutexA.

Outcome:

1) **Module Dependency Tree View** - shows EXE file and 2 Windows libraries it uses.
2) **Parent Imports** - functions the EXE file uses from the library.
3) **_stricmp** - indicator of a program performs a string comparison.
4) **FindNextFileA** and **FindFirstFileA -** Functions to manipulate files, malware searches through the system.
5) **Linking by Ordinal** - a method of accessing functions by their unique identifier (ordinal number) instead of their text name, can't easily see what functions are in use.
6) **Berkeley Sockets (accept, bind, connect)** - functions used for networking, suggest that the malware performs some networking functions, such as connecting to a server and opening a listening port.
7) **CreateProcessA** - a specific function within the Windows API (Application Programming Interface) used to create a new process and its primary thread
8) **Sleep** - a standard exported or imported function used to pause the execution of the current thread for a specified duration.
9) **OpenMutexA** - an imported function name indicating that the executable or DLL being analyzed uses the Windows API function OpenMutex to open an existing named mutex object.

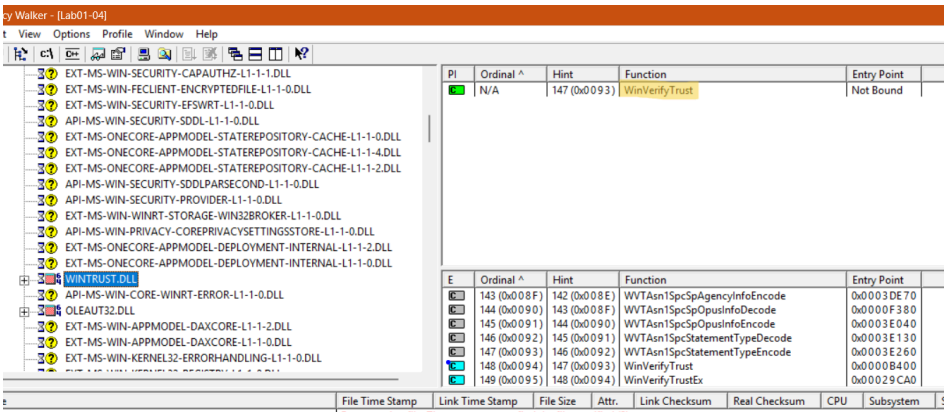
EXTRA LAB01-04 TASK:

1. **Flag PMA 101.5: Find the Downloaded File**



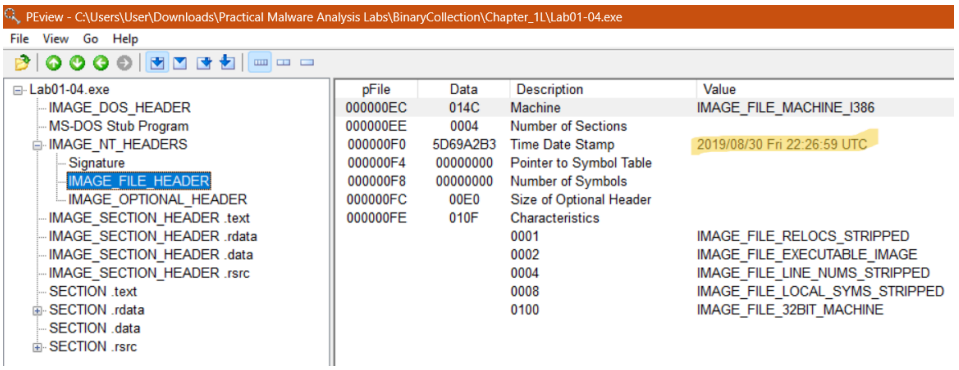
File name: http://www.practicalmalwareanalysis.com/updater.exe

2. **Flag PMA 101.6: Find the Imported Function**



Function name: **WinVerifyTrust.**

3. **Flag PMA 101.7: Find the Datestamp**



Date when sample **Lab01-04.exe** was compiled**: 2019/08/30**

6. Overall Interpretation — How Malware Was Detected

Using static analysis tools, we identified the following:

I. Confirmation of Malicious Status

II. Behavioral Indicators

III. Indicators of Compromise (IoCs)

IV. Malware Architecture

a. Evidence Suggests

1. Networking Capability (e.g., from WS2_32.DLL or wininet.dll)

- **Evidence (Common Functions):** `connect`, `send`, `recv`, `InternetOpenA`, `InternetConnectA`.
- **Implication:** The presence of these functions is strong evidence that the malware can **initiate or receive network connections**. This is typically required to:
  - Communicate with a **Command and Control (C2) server**.
  - Transmit stolen data.
  - Download additional malicious components.

2. User Interaction (e.g., from USER32.DLL)

- **Evidence (Common Functions):** `MessageBoxA`, `DialogBoxParamA`, `CreateWindowExA`.
- **Implication:** The ability to call these functions means the malware can **interact with the user interface**. The function `MessageBoxA` is frequently used by malware to display a **fake error message** (e.g., "The file is corrupted") to distract the user while the actual malicious activity is happening in the background.

3. File and Process Control (e.g., from KERNEL32.DLL - as discussed previously)

- **Evidence (Common Functions):** `CreateProcessA`, `OpenMutexA`, `FindFirstFileA`, `FindNextFileA`.
- **Implication:** This evidence indicates core functions like **process creation**, **system searching**, and **controlling execution** (using mutexes).